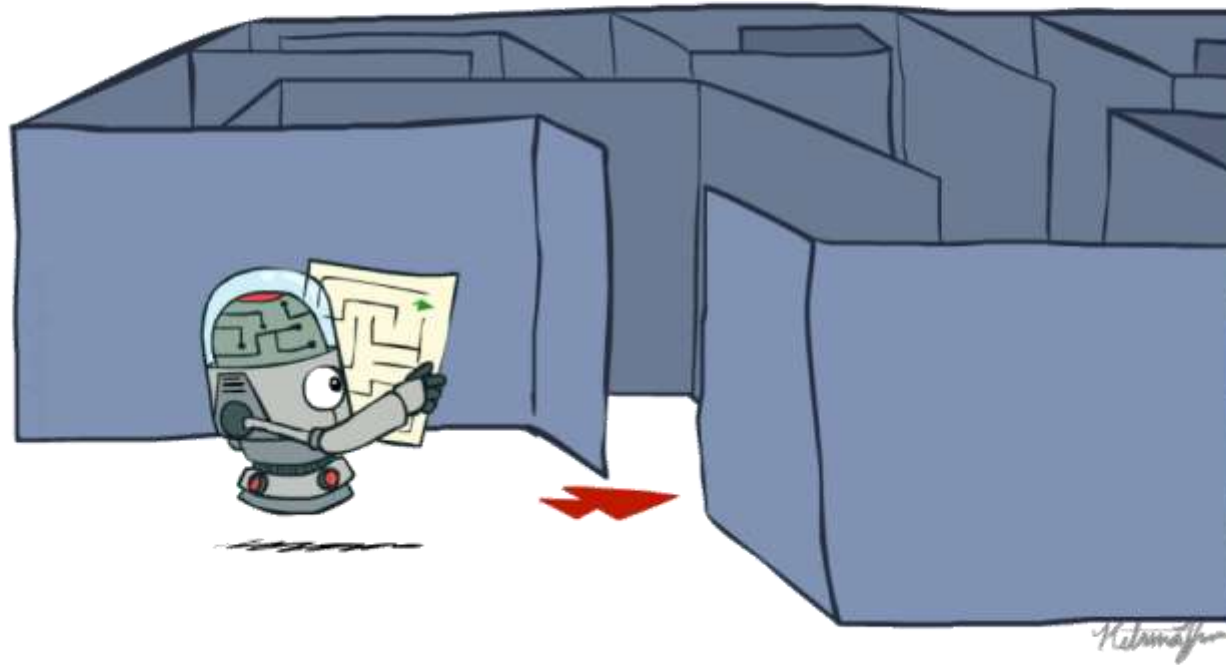


AIT2004: Cơ sở Trí tuệ nhân tạo

Các giải thuật tìm kiếm cơ bản



[Sử dụng một số slides của Dan Klein và Pieter Abbeel từ CS188 Intro to AI at UC Berkeley]

Học liệu

- Đinh Mạnh Tường, *Trí tuệ nhân tạo – Cách tiếp cận hiện đại*, NXB Khoa học kỹ thuật, 2024
 - Chương 3. Các chiến lược tìm kiếm mù
- Wolfgang Ertel, *Introduction to Artificial Intelligence (2nd ed)*, Springer 2017
 - Chapter 6. Search, Games and Problem Solving
- Russell & Norvig, *AI: A Modern Approach, 4th ed.*, Pearson 2020
 - Chapter 3. Solving Problems by Searching

Bài toán tìm kiếm

- Nhiều bài toán được quy về dạng bài toán tìm kiếm
 - Trò chơi: chuỗi các nước đi
 - Tìm đường đi: ngắn nhất, nhanh nhất
- Mô hình hóa thành không gian các trạng thái
- Lời giải
 - Đường đi từ bắt đầu đến kết thúc
=> Cây tìm kiếm
- Chú ý: tìm kiếm $\neq$ truy vấn thông tin



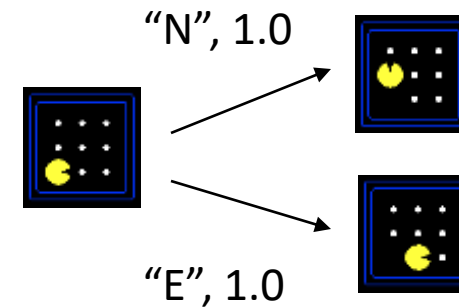
Bài toán tìm kiếm

- Định nghĩa bài toán tìm kiếm:

- Không gian trạng thái

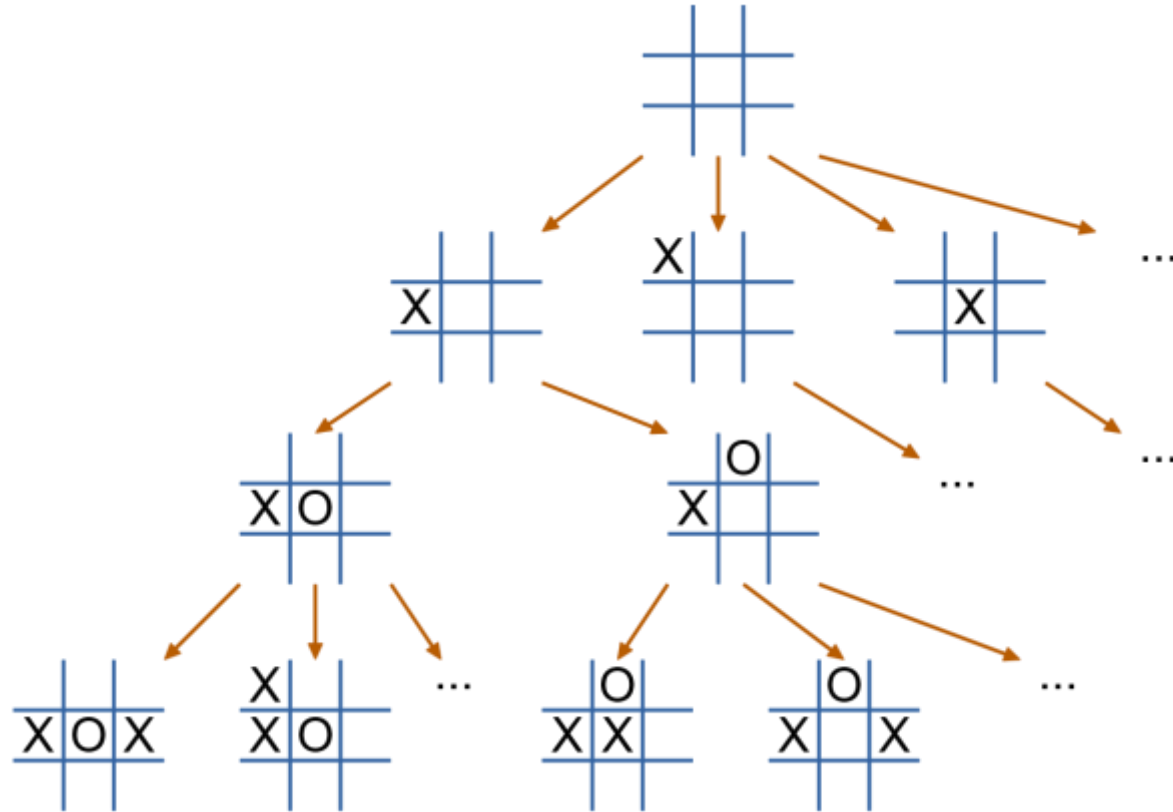


- Hàm chuyển trạng thái
(kèm hành động + chi phí)



- Trạng thái bắt đầu và hàm kiểm tra trạng thái mục tiêu
- Giải pháp = dãy hành động (kế hoạch) chuyển từ trạng thái bắt đầu đến trạng thái mục tiêu

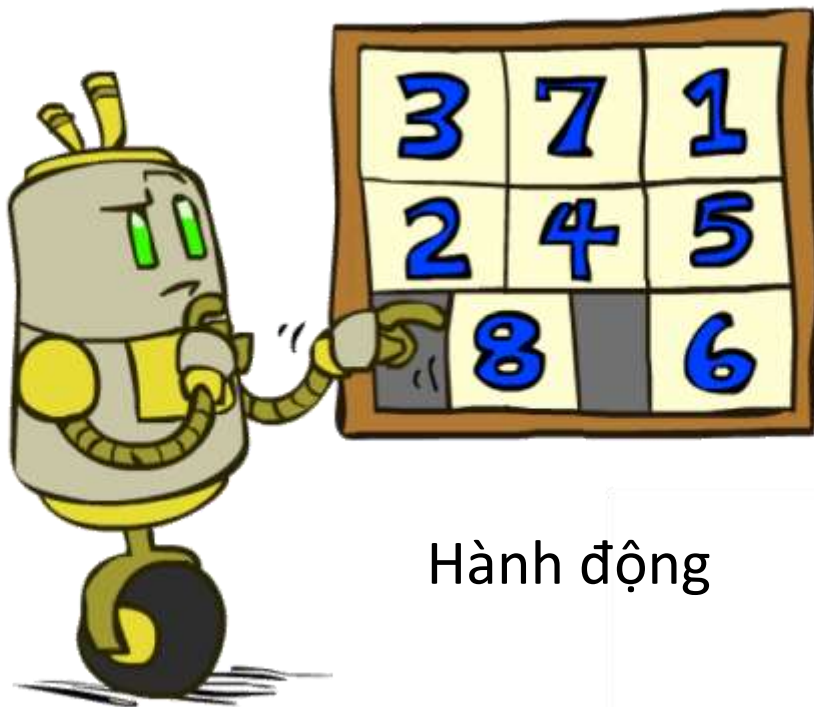
Ví dụ: cờ ca rô



Ví dụ: Trò chơi đẩy ô số

7	2	4
5		6
8	3	1

Trạng thái bắt đầu



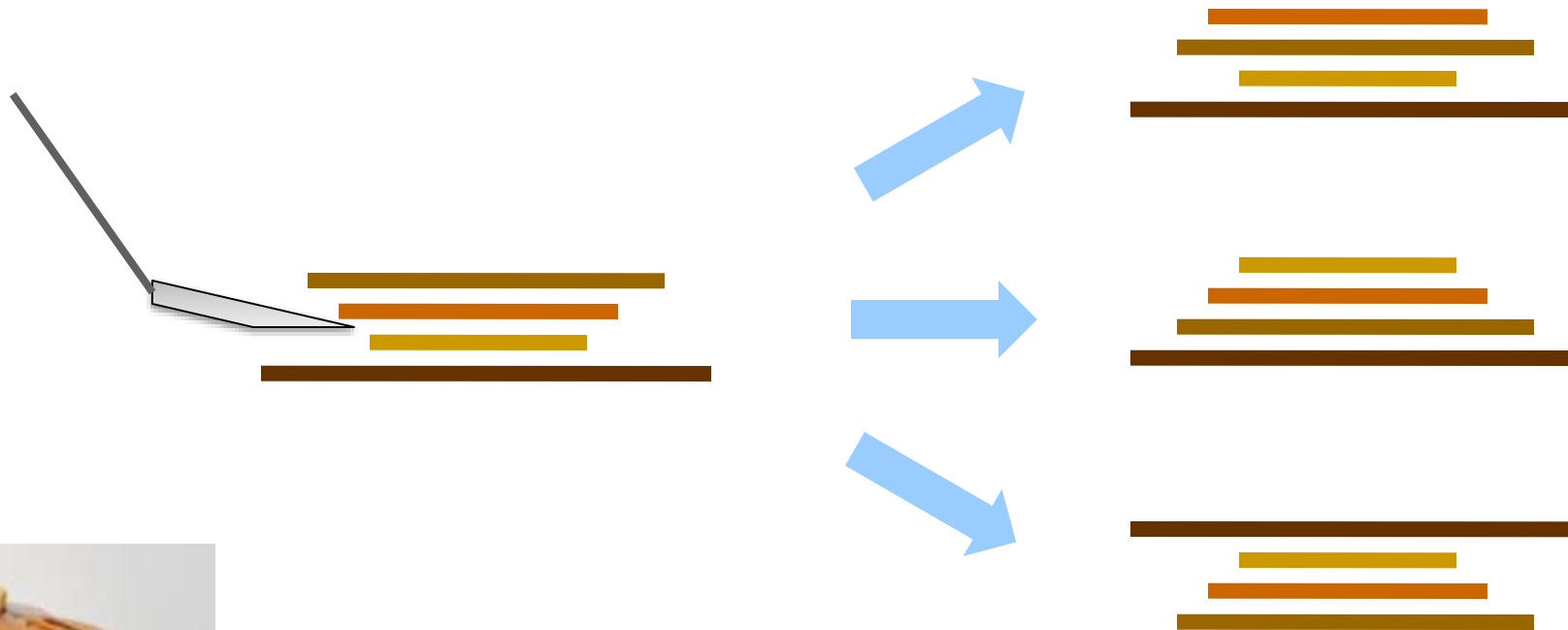
Hành động

	1	2
3	4	5
6	7	8

Trạng thái mục tiêu

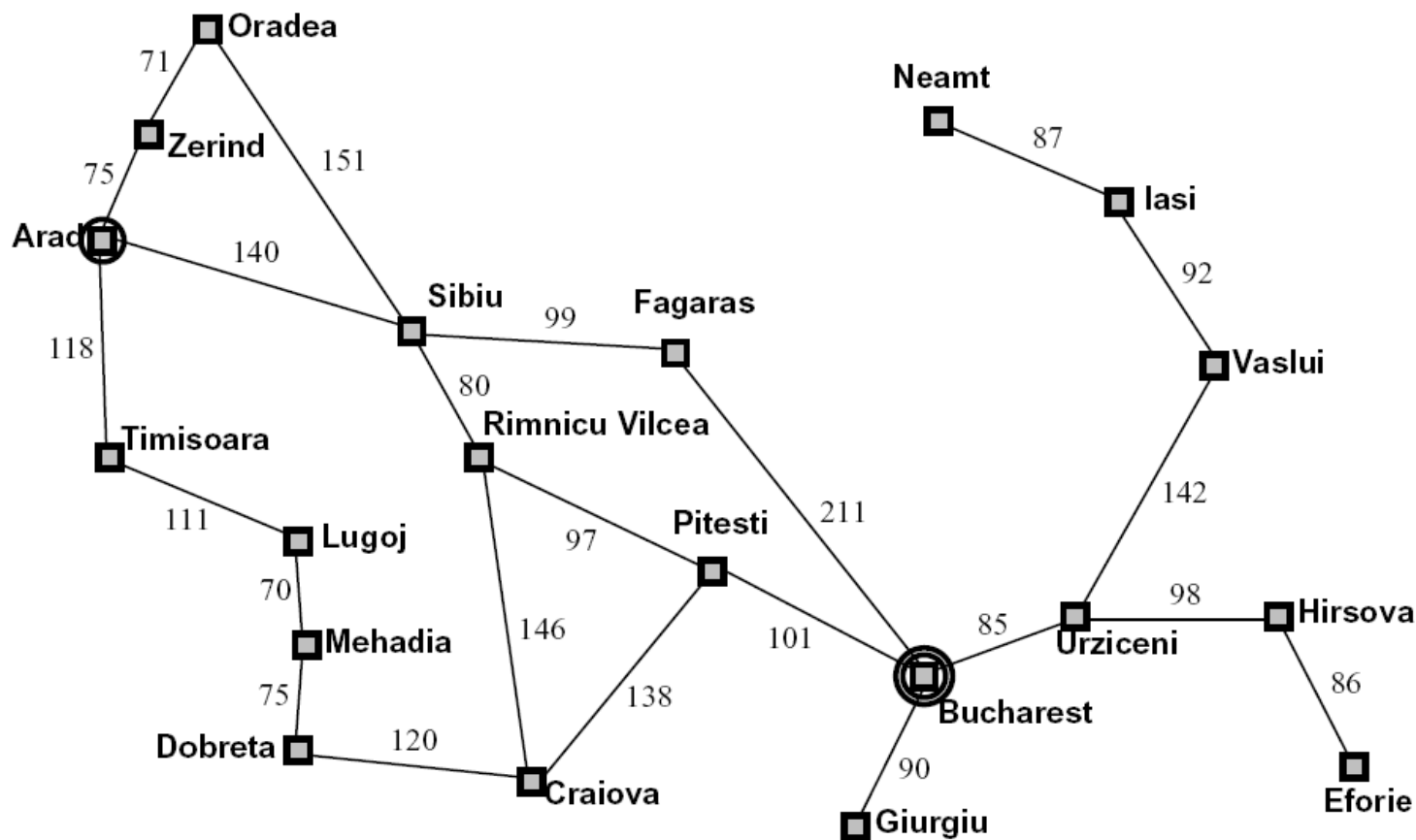
- Trạng thái là gì?
- Có bao nhiêu trạng thái?
- Hành động là gì?
- Có bao nhiêu trạng thái đến được từ trạng thái bắt đầu?
- Chi phí là gì?

Ví dụ: Lật chồng bánh pancake



Chi phí = tổng số lớp bánh được lật

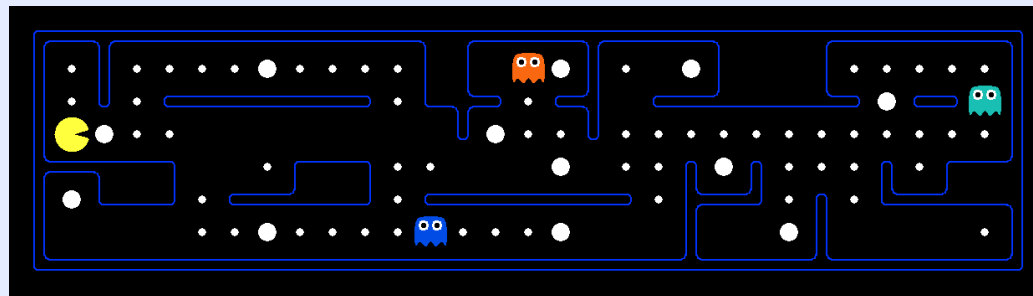
Ví dụ: Du lịch Romania



- Không gian trạng thái:
 - Các thành phố
- Hàm chuyển trạng thái:
 - Hành động = chọn đường
 - Chi phí = khoảng cách
- Trạng thái bắt đầu:
 - Arad
- Hàm kiểm tra mục tiêu:
 - Hỏi state == Bucharest?
- Câu hỏi: Giải pháp = ?

Không gian trạng thái

Trạng thái của môi trường / thế giới bao gồm mọi chi tiết về môi trường / thế giới



Trạng thái tìm kiếm chỉ gồm các chi tiết cần thiết cho lập kế hoạch (trừu tượng hoá)

■ Bài toán: Tìm đường đi

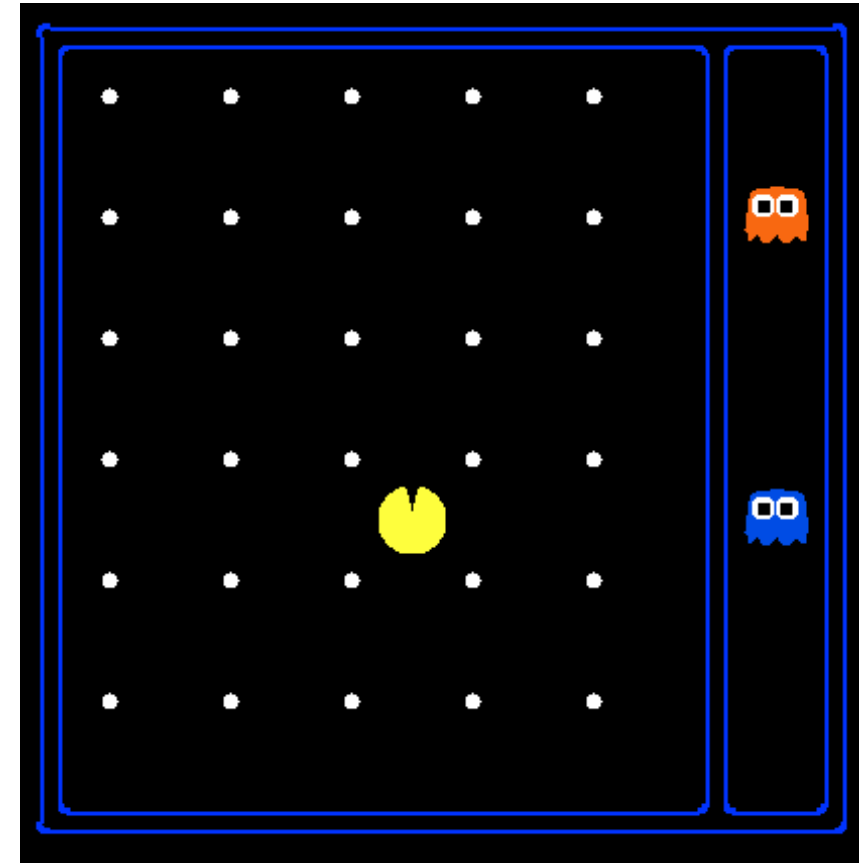
- Trạng thái: Vị trí (x,y)
- Hành động: N – S – E – W
- Hàm chuyển: cập nhật vị trí
- Hàm kiểm tra: $(x,y)=\text{END}$?

■ Bài toán: Ăn hết bánh

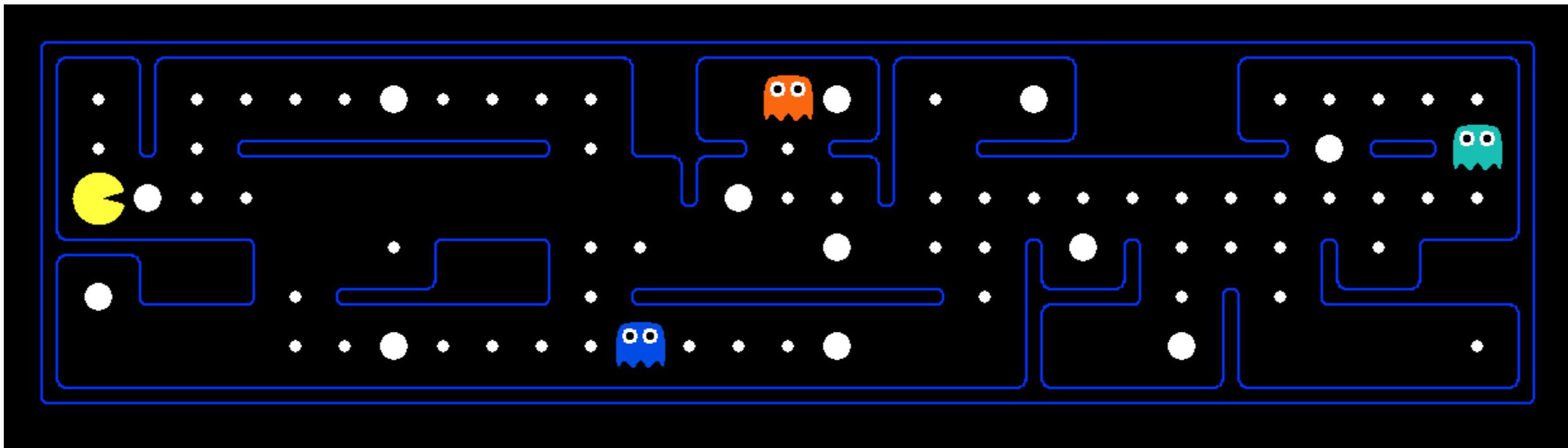
- Trạng thái: Vị trí (x,y) + danh sách các ô có bánh
- Hành động: N – S – E – W
- Hàm chuyển: cập nhật vị trí và danh sách các ô có bánh
- Hàm kiểm tra : danh sách rỗng ?

Kích thước của không gian trạng thái

- Trạng thái của môi trường:
 - Số vị trí của Pac-man: 120
 - Số ô có bánh: 30
 - Số vị trí của ma: 12
 - Hướng của Pac-man: 4
- Số lượng trạng thái
 - Trạng thái của môi trường ?
 $120 \times (2^{30}) \times (12^2) \times 4$
 - Trạng thái cho bài toán tìm kiếm ?
120
 - Trạng thái cho bài toán ăn hết bánh ?
 $120 \times (2^{30})$



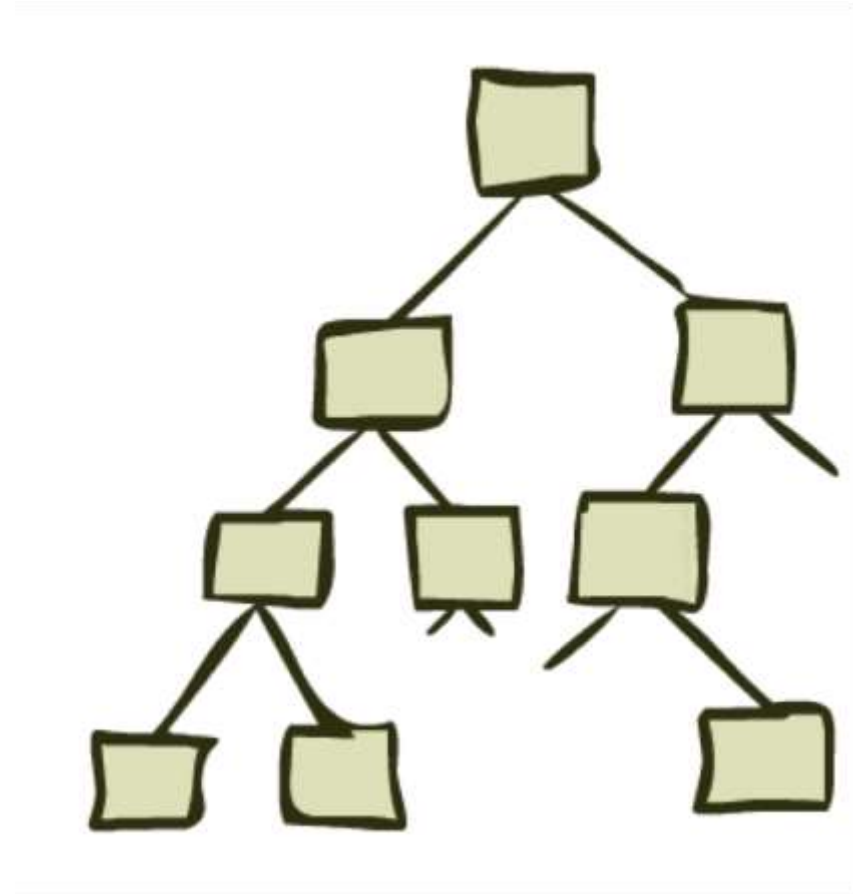
Bài tập: Đường đi an toàn



- Bài toán: ăn hết bánh và giữ cho các con ma sợ
- Trạng thái tìm kiếm gồm những gì ?
 - (vị trí Pac-man, danh sách vị trí bánh, danh sách vị trí thuốc, thời gian ma sợ còn lại)

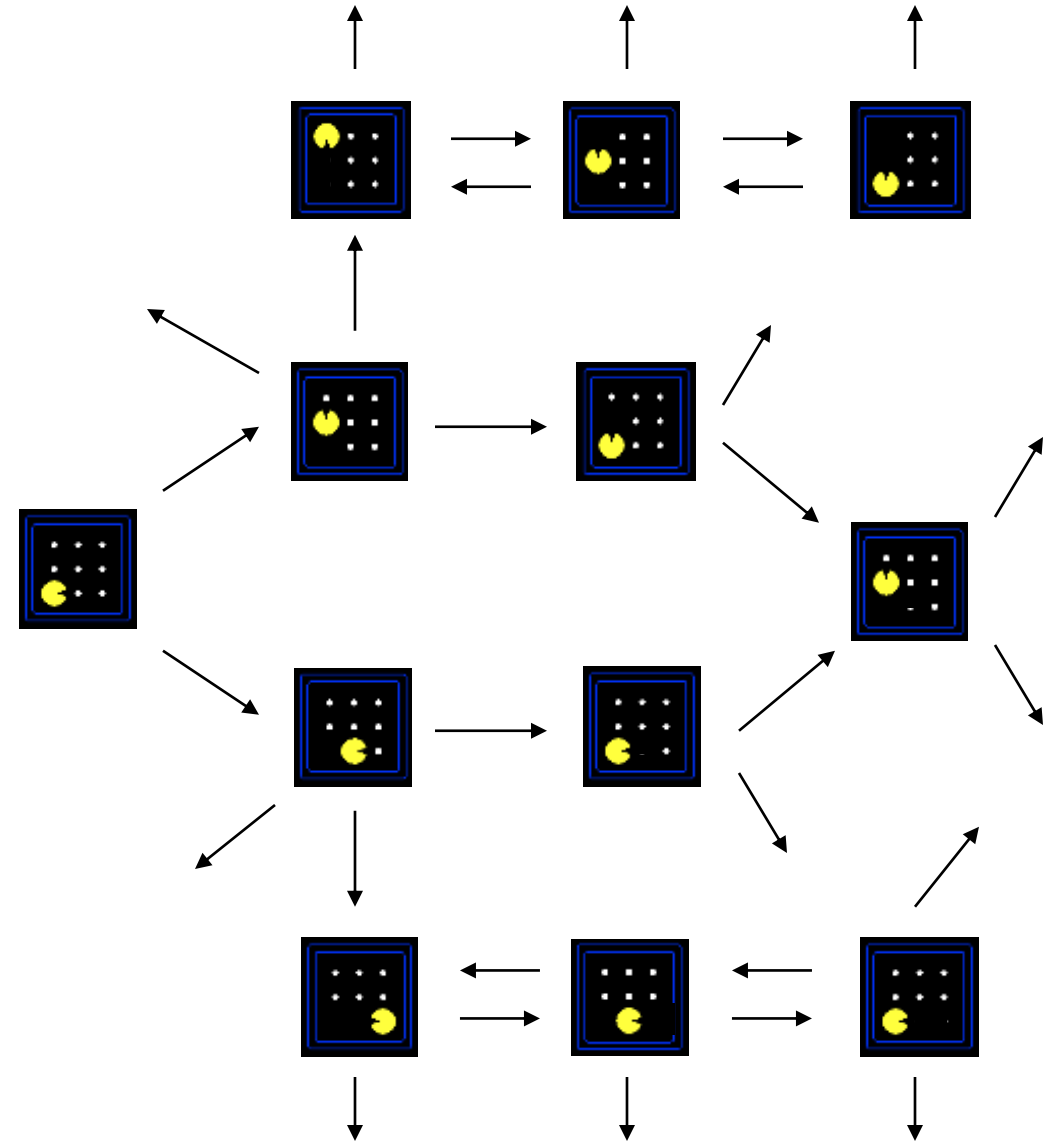
Đồ thị không gian trạng thái và cây tìm kiếm

- Có thể hình dung không gian trạng thái của bài toán như một đồ thị
- Việc tìm lời giải là duyệt đồ thị theo một chiến lược nhất định, hay nói cách khác là tạo ra cây tìm kiếm trên đồ thị



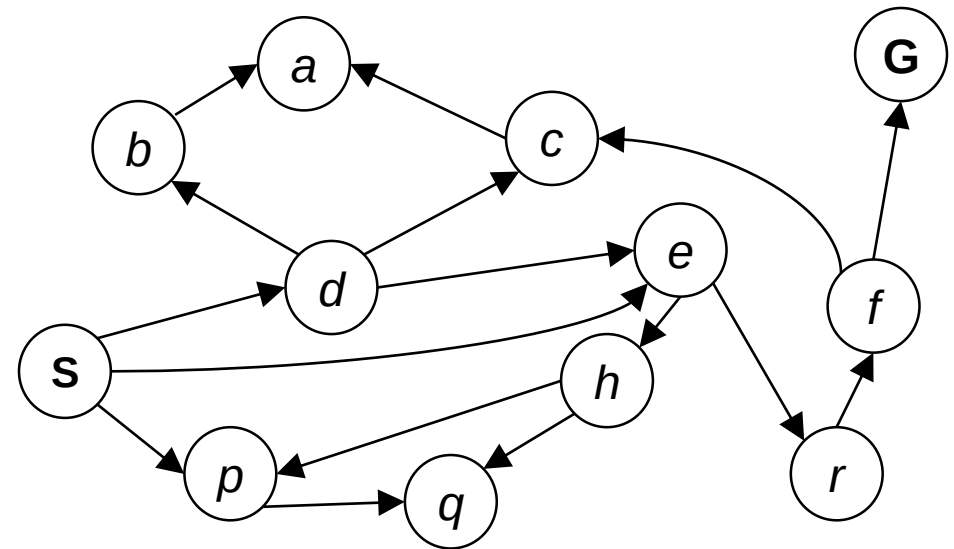
Đồ thị không gian trạng thái

- **Đồ thị không gian trạng thái**: biểu diễn toán học của bài toán tìm kiếm
 - Đỉnh = Trạng thái
 - Cạnh = hàm chuyển
 - Hàm kiểm tra = tập các đỉnh mục tiêu
- Trong đồ thị không gian trạng thái, mỗi trạng thái xuất hiện **duy nhất một lần**!
- Hiếm khi xây dựng đồ thị này (do quá lớn) mà chúng ta dùng nó để suy nghĩ về bài toán



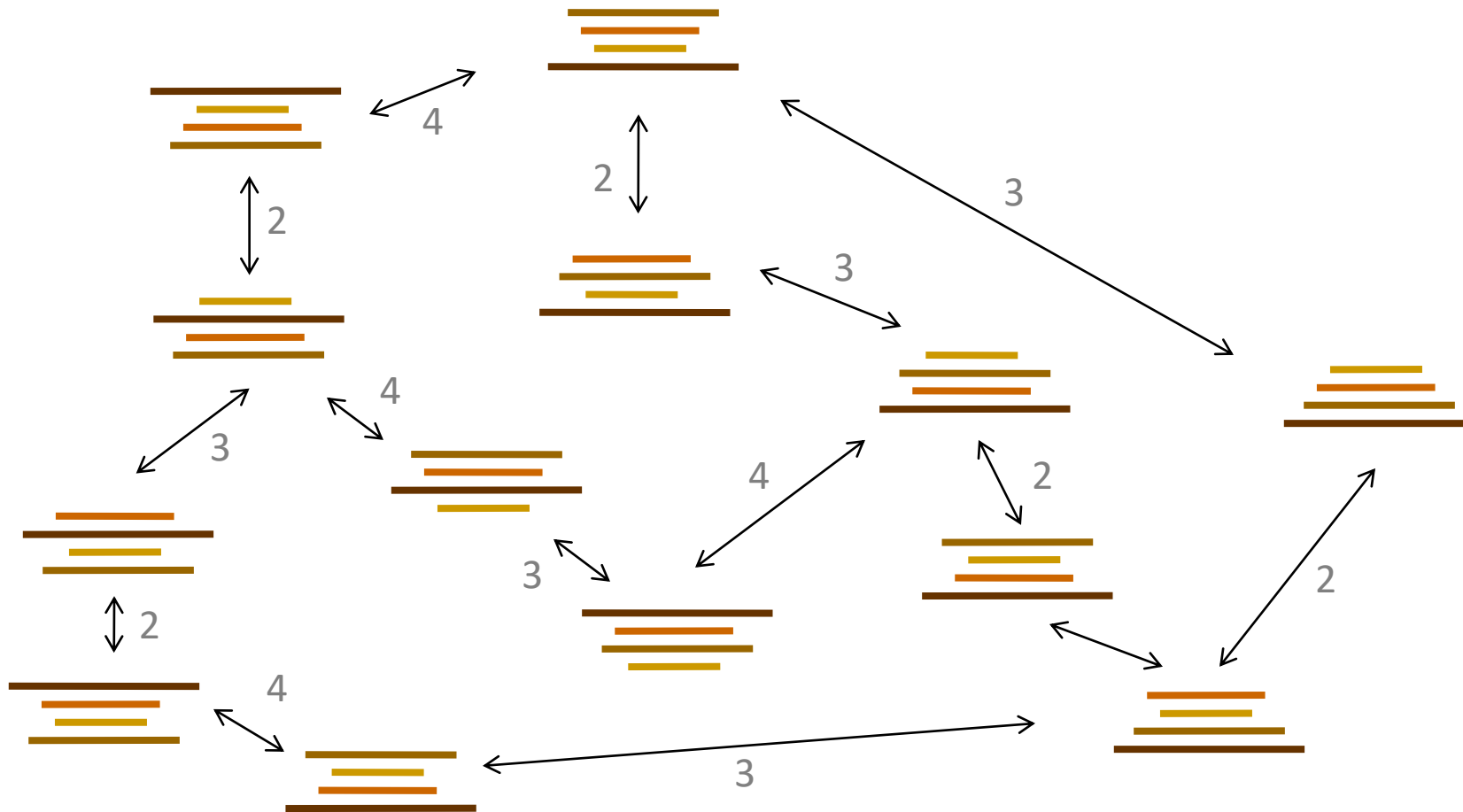
Đồ thị không gian trạng thái

- **Đồ thị không gian trạng thái**: biểu diễn toán học của bài toán tìm kiếm
 - Đỉnh = Trạng thái
 - Cạnh = hàm chuyển
 - Hàm kiểm tra = tập các đỉnh mục tiêu
- Trong đồ thị không gian trạng thái, mỗi trạng thái xuất hiện **duy nhất một lần**!
- Hiếm khi xây dựng đồ thị này (do quá lớn) mà chúng ta dùng nó để suy nghĩ về bài toán

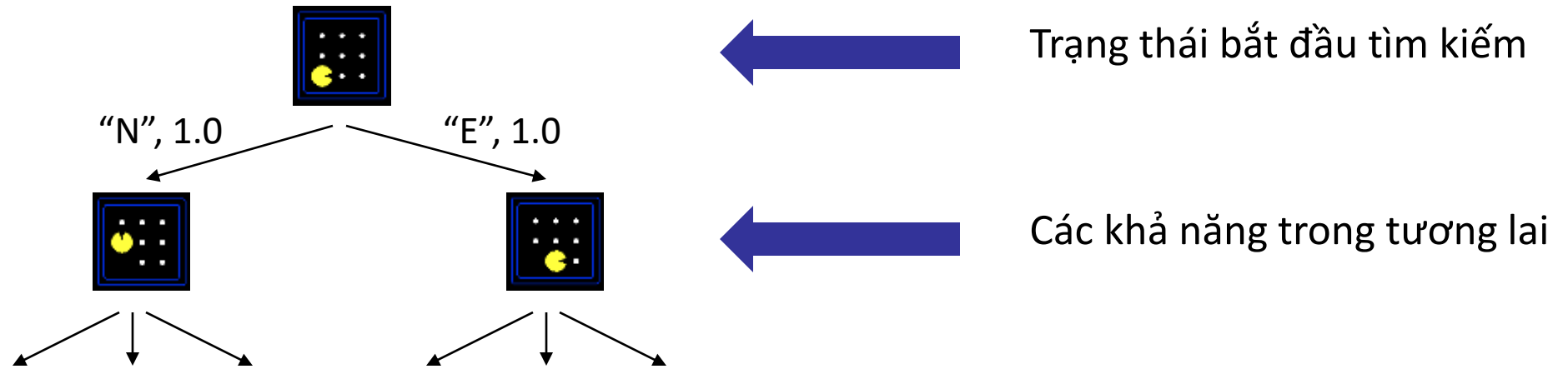


Ví dụ: Lật chồng bánh pancake

Đồ thị không gian trạng thái với chi phí hành động = số bánh được lật



Cây tìm kiếm

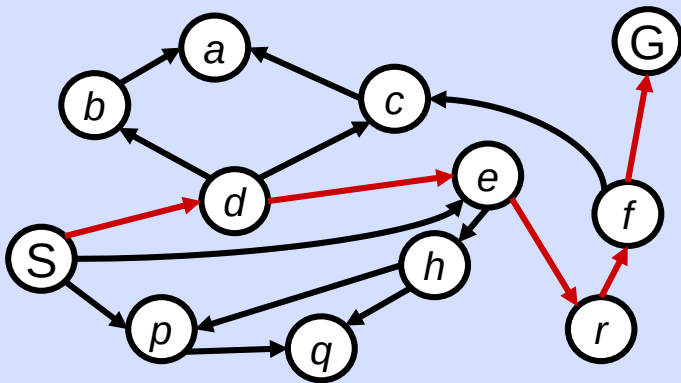


- **Cây tìm kiếm:**

- Cây của các kế hoạch và kết quả: Câu hỏi “Nếu ... thì ...”
- Trạng thái bắt đầu = gốc cây
- Các nút con = các phép chuyển trạng thái
- Mỗi nút con = một kế hoạch đến một trạng thái nào đó
- **Hầu như không thể xây dựng toàn bộ cây tìm kiếm**

Đồ thị không gian trạng thái $\neq$ Cây tìm kiếm

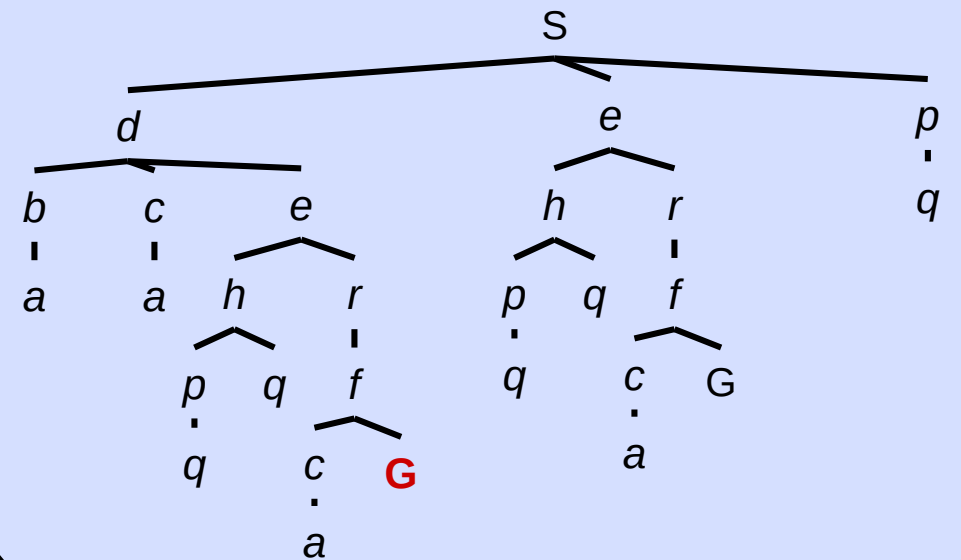
Đồ thị không gian trạng thái



Mỗi nút trong cây tìm kiếm tương đương với một đường đi trong đồ thị không gian trạng thái

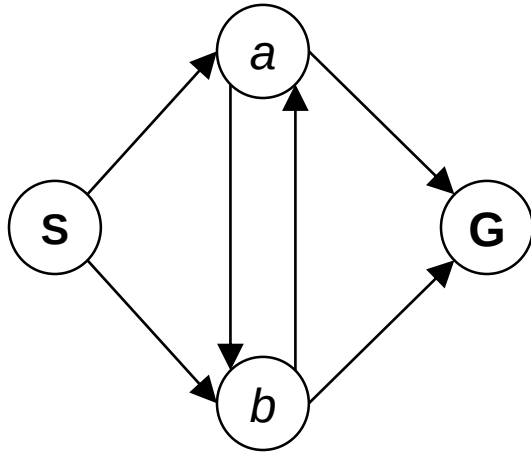
Xây dựng đồ thị và cây tìm kiếm “theo nhu cầu” và càng ít càng tốt

Cây tìm kiếm

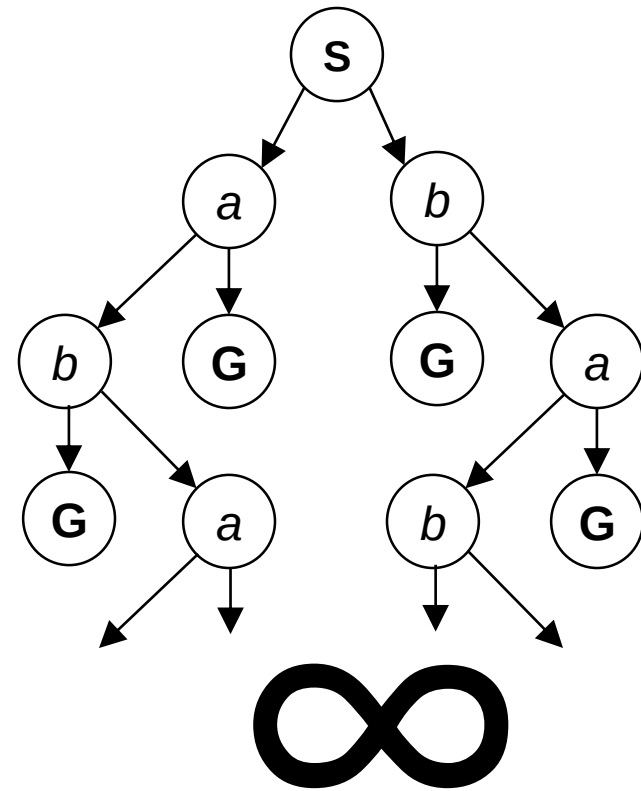


Đồ thị không gian trạng thái $\neq$ Cây tìm kiếm

Xét đồ thị gồm 4 đỉnh sau:

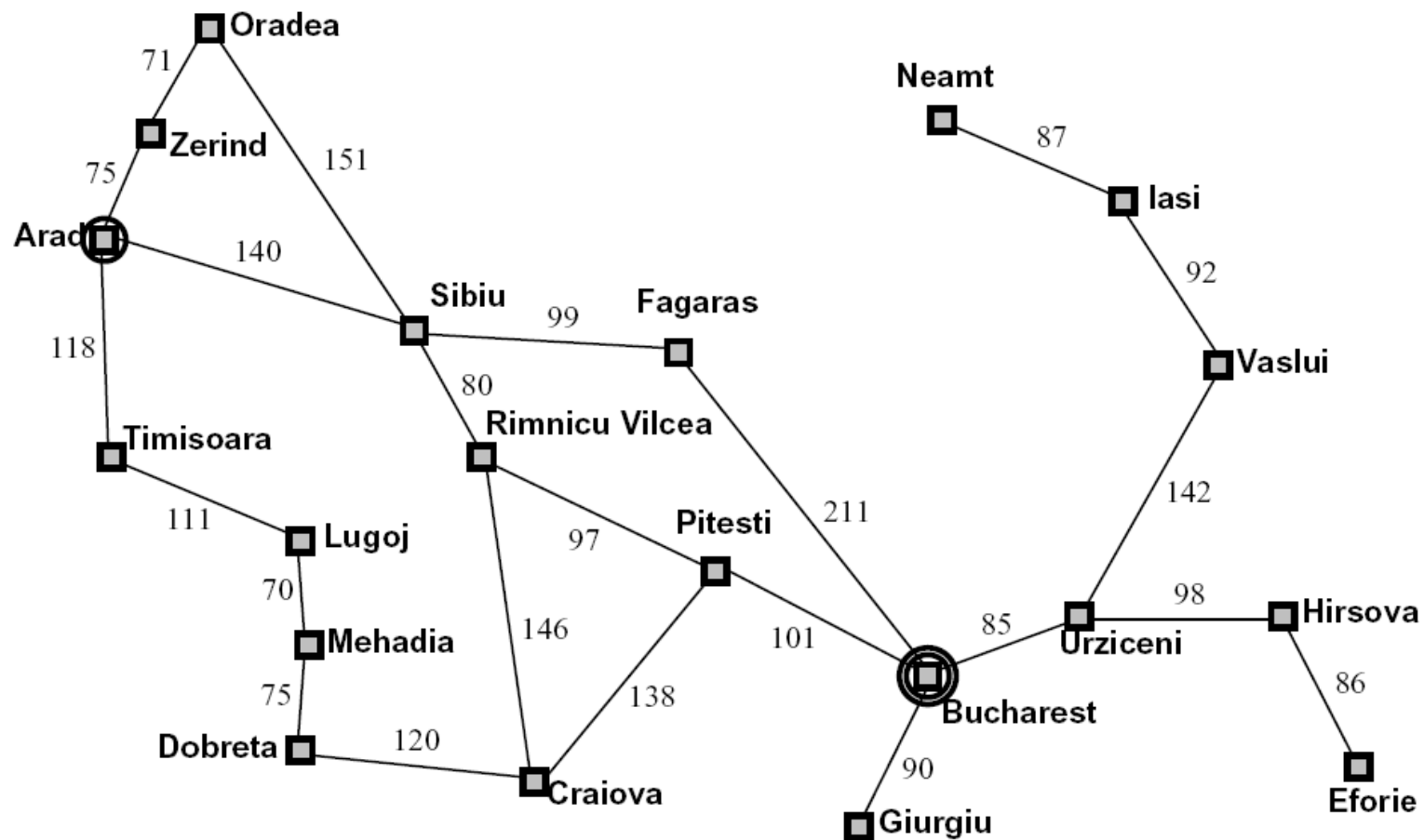


Độ sâu cây tìm kiếm xuất phát từ S?

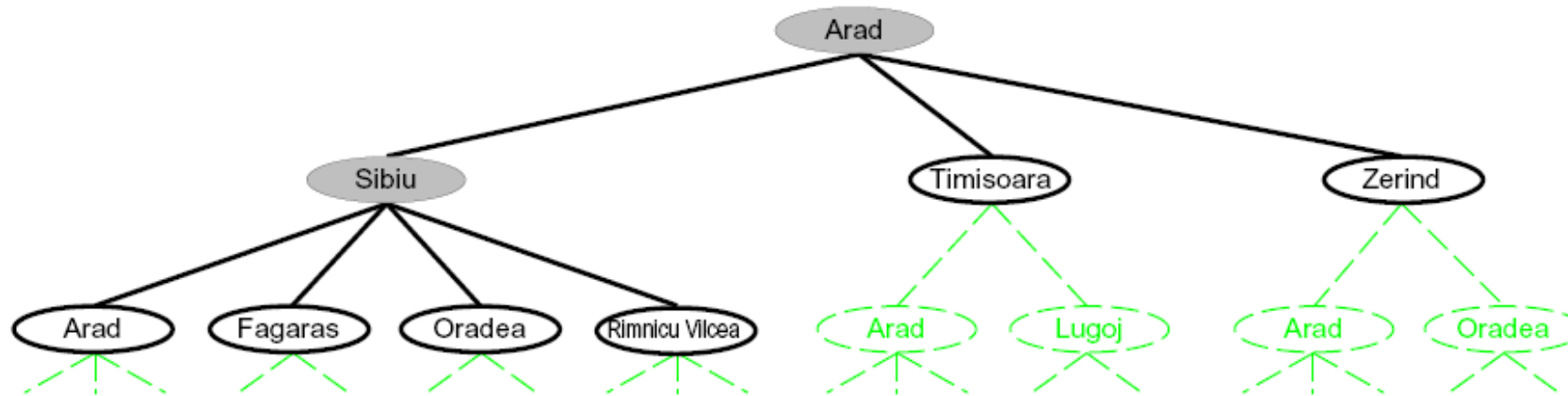


Lưu ý: có rất nhiều cây con bị lặp lại trong cây tìm kiếm

Ví dụ: Du lịch Romania



Cây tìm kiếm



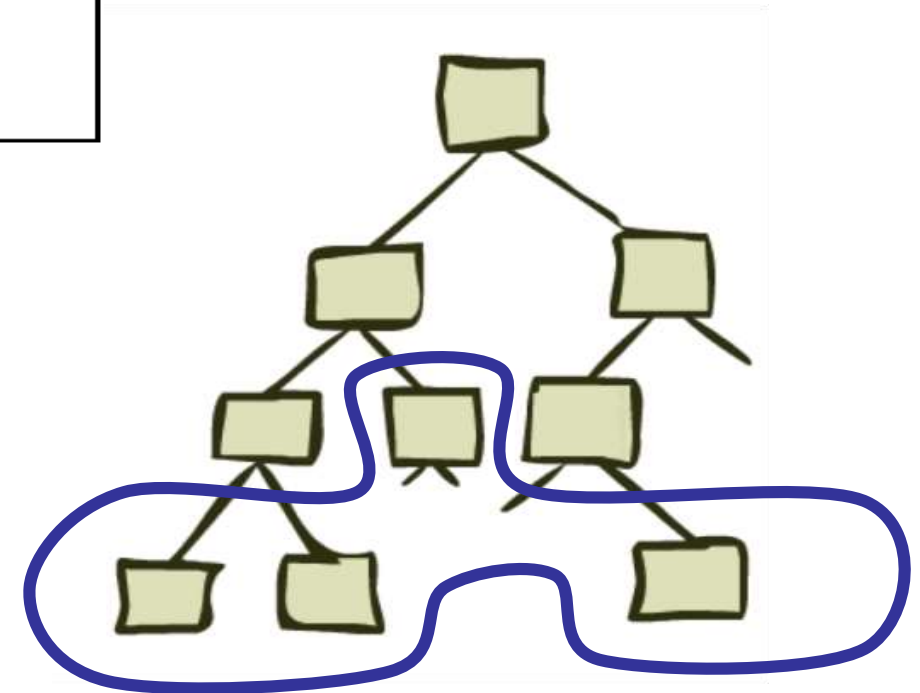
■ Tìm kiếm:

- Mở rộng các kế hoạch (đại diện bởi các nút của cây)
- Quản lý vùng “rìa” hoặc “biên” tìm kiếm bao gồm các kế hoạch đang quan tâm
- Mở rộng càng ít càng tốt

Thuật toán tìm kiếm tổng quát

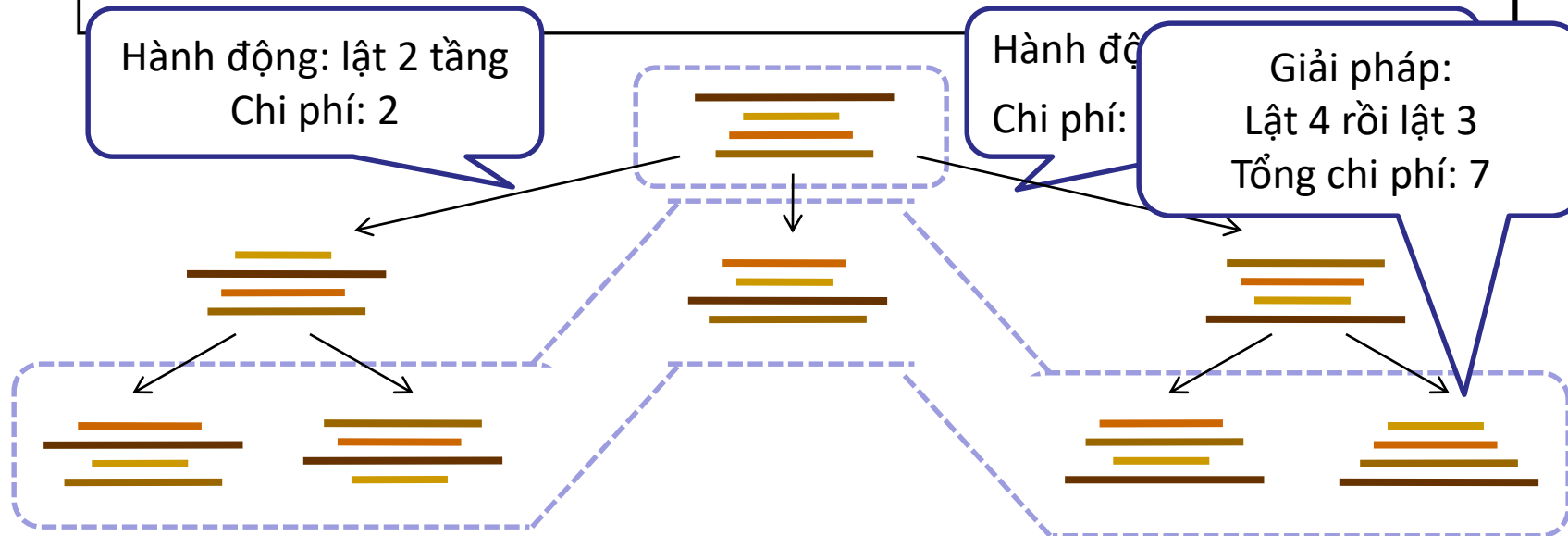
```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

- Các điểm cần chú ý:
 - Biên tìm kiếm (fringe, frontier)
 - Mở rộng biên
 - Chiến lược khám phá
- Câu hỏi: Nốt biên nào cần được khám phá?



Thuật toán tìm kiếm tổng quát

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```



Các chiến lược tìm kiếm (khám phá)

- Tìm kiếm theo chiều sâu (DFS)
- Tìm kiếm theo chiều rộng (BFS)
- Tìm kiếm độ sâu tăng dần
- Tìm kiếm chi phí đều (UCS)

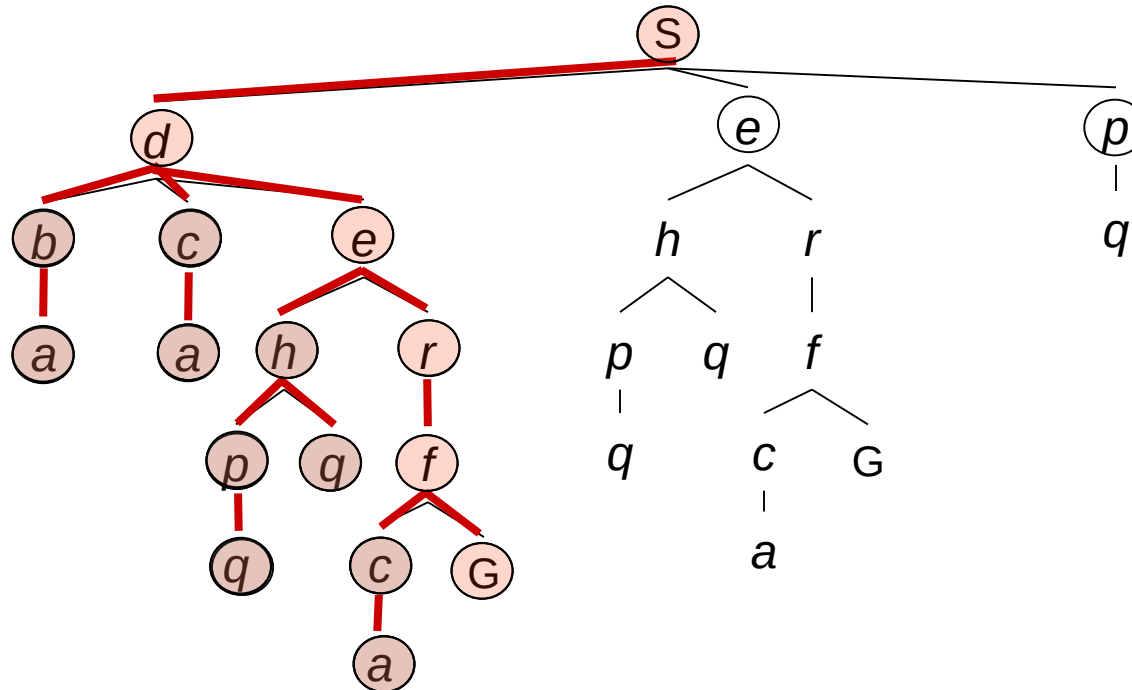
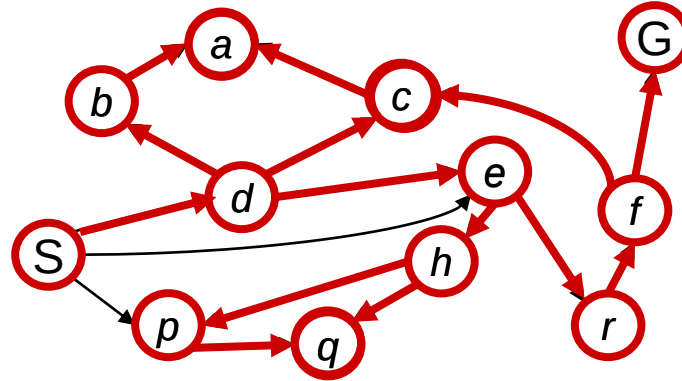
Tìm kiếm theo chiều sâu (DFS)



Tìm kiếm theo chiều sâu (DFS)

*Chiến lược: mở rộng nút
sâu nhất trong cây*

Cài đặt: Biên = Ngăn xếp



Các tính chất của thuật toán tìm kiếm

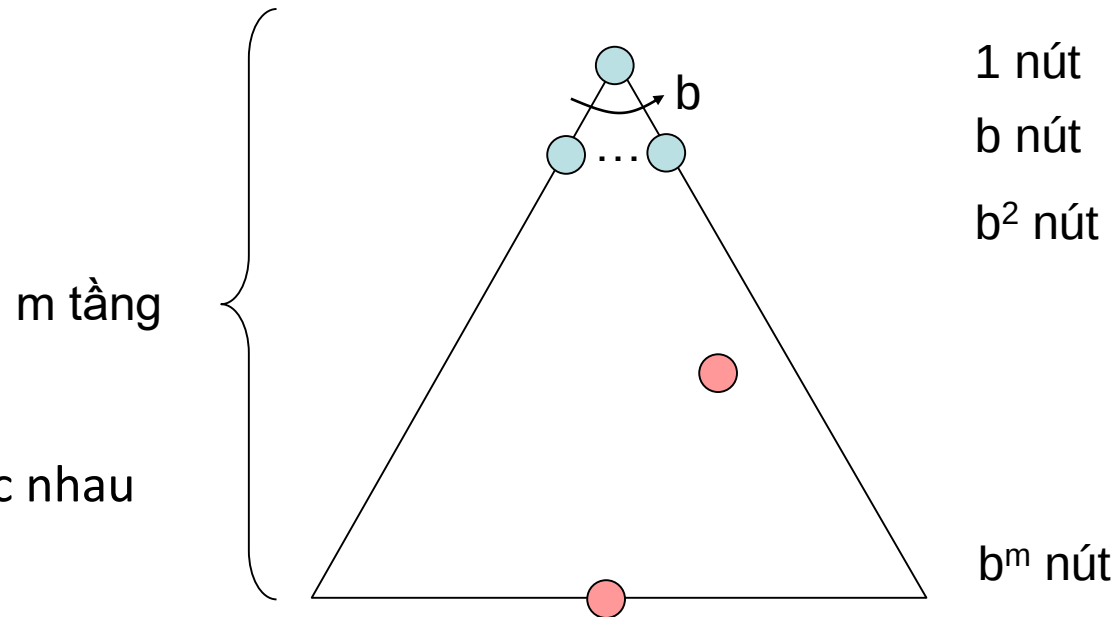
- **Đầy đủ:** Luôn tìm được giải pháp nếu tồn tại ?
- **Tối ưu:** Đảm bảo tìm được giải pháp có chi phí tốt nhất ?
- **Độ phức tạp thời gian ?**
- **Độ phức tạp không gian ?**

- **Các yếu tố của cây tìm kiếm:**

- b : hệ số **rẽ nhánh**
- m : **độ sâu** lớn nhất
- Giải pháp có thể nằm ở các độ sâu khác nhau

- **Số lượng nút trong cây tìm kiếm?**

- $1 + b + b^2 + \dots + b^m = O(b^m)$



Tính chất của DFS

- DFS mở rộng các nút nào trong cây ?

- Các nhánh bên trái của cây.
- Có thể duyệt toàn bộ cây !
- Nếu m hữu hạn, cần $O(b^m)$ bước

- Bộ nhớ cho biên tìm kiếm ?

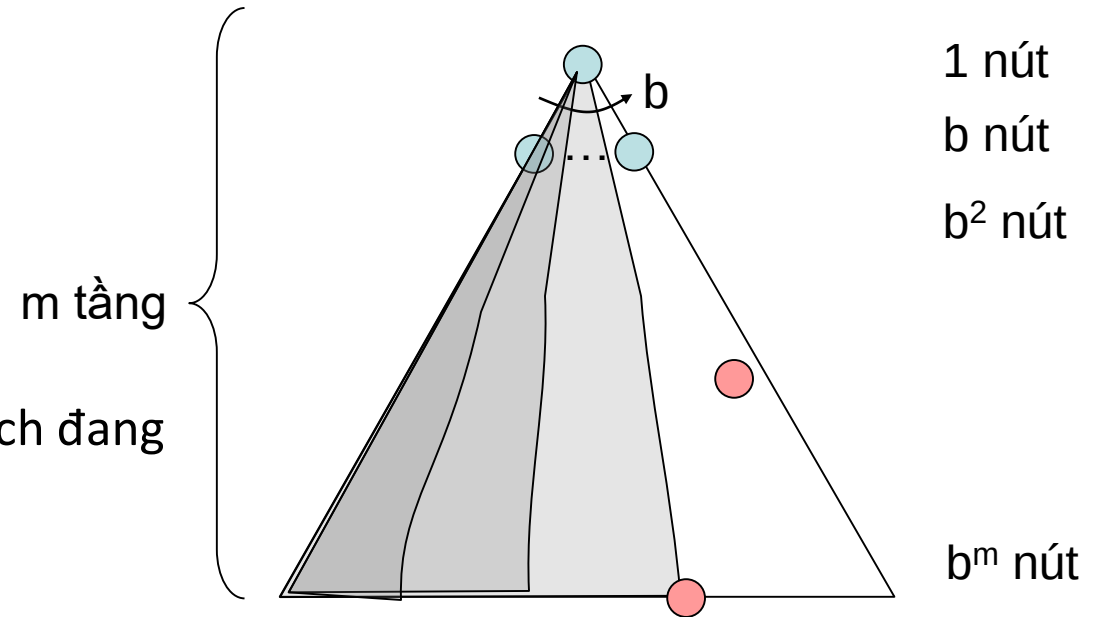
- Chỉ lưu các anh em của các nút dọc theo kế hoạch đang xét $\rightarrow O(bm)$

- Đầy đủ ?

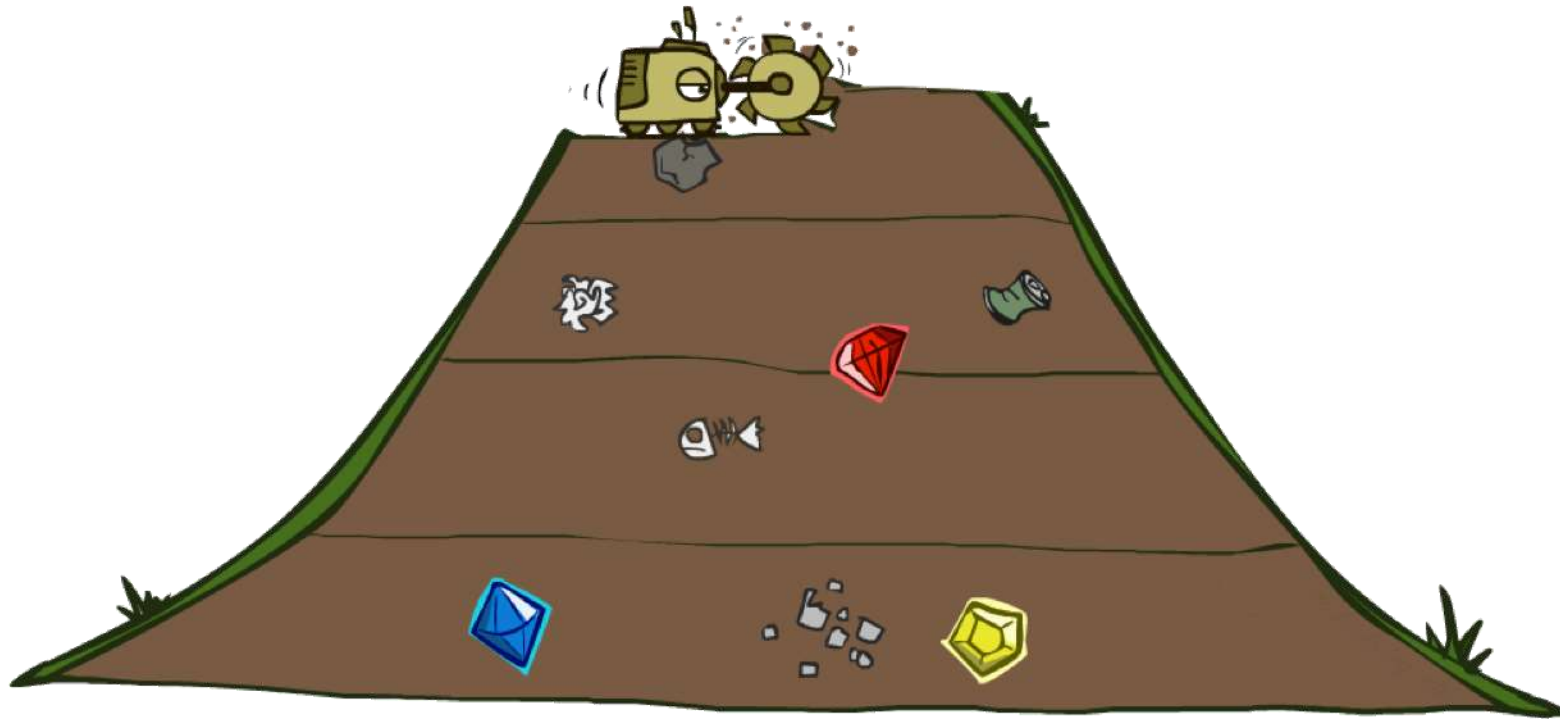
- Do m có thể vô hạn $\rightarrow$ chỉ đầy đủ khi loại bỏ được chu trình

- Tối ưu ?

- Không, DFS tìm được mục tiêu bên trái nhất trong cây



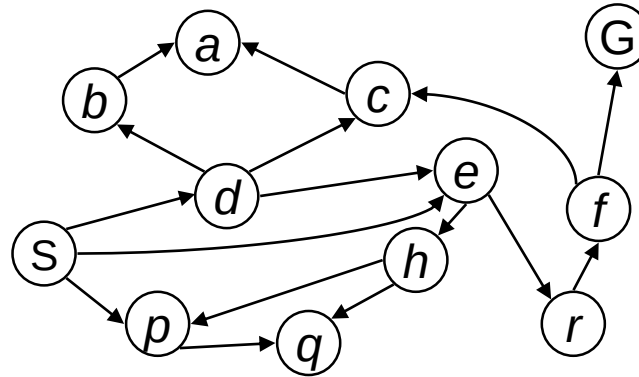
Tìm kiếm theo chiều rộng (BFS)



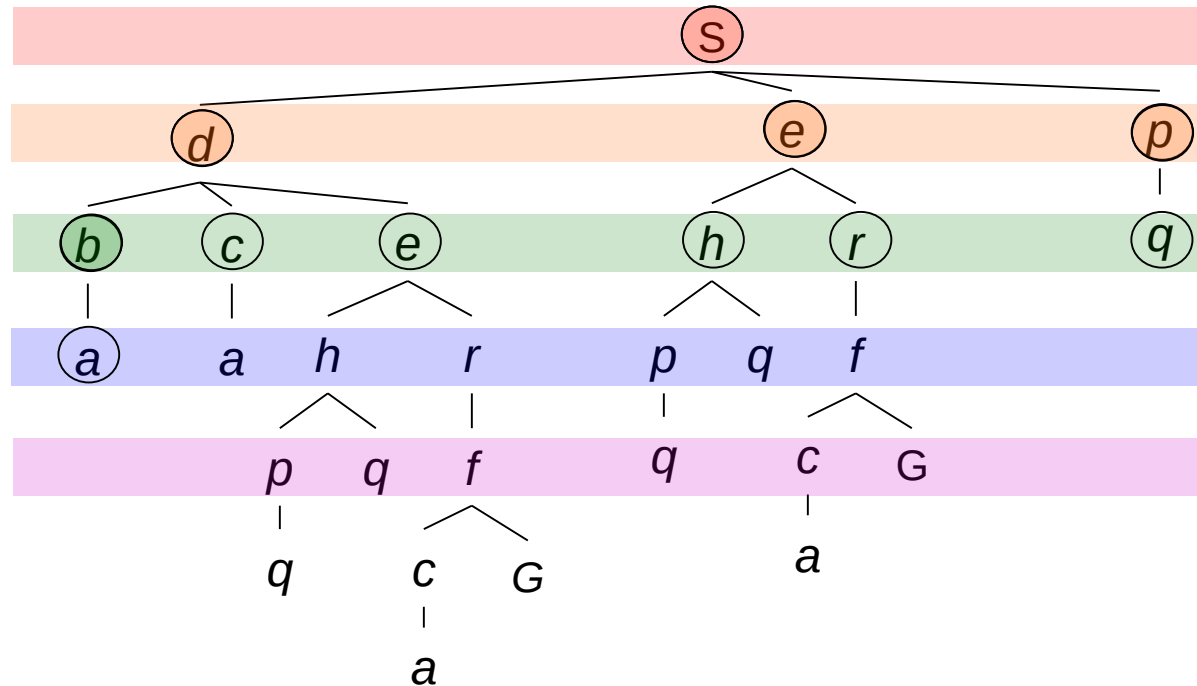
Tìm kiếm theo chiều rộng (BFS)

Strategy: Mở rộng nút
nông nhất trong cây

Cài đặt: Biên = Hàng đợi

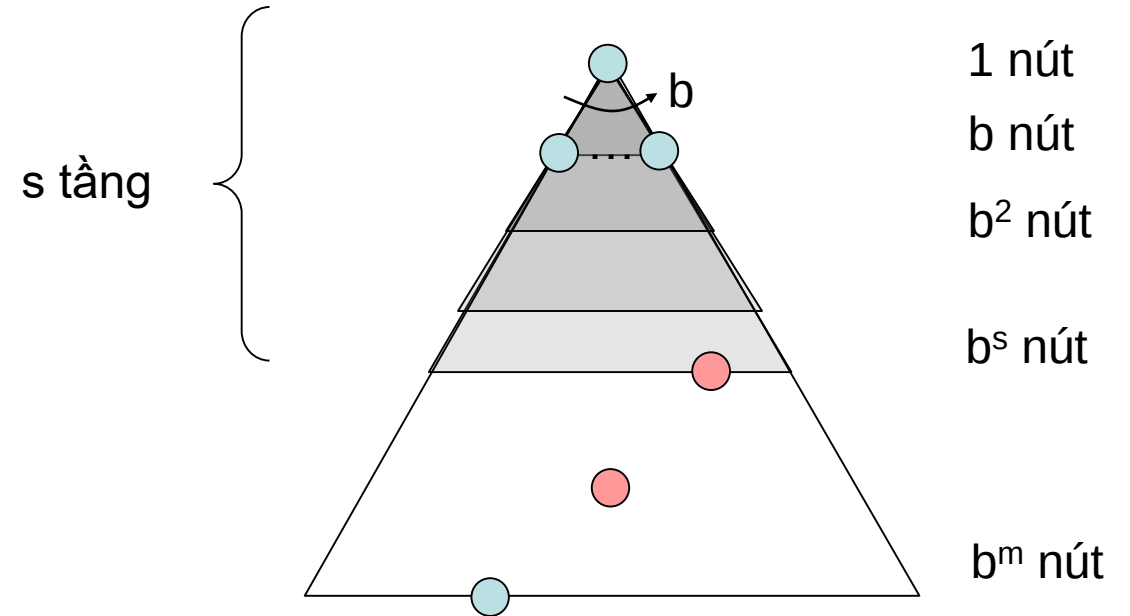


Search
Tiers



Tính chất của BFS

- BFS mở rộng các nút nào ?
 - Các kế hoạch ngắn hơn giải pháp
 - Nếu giải pháp ngắn nhất ở độ sâu s
 - Độ phức tạp thời gian $O(b^s)$
- Bộ nhớ cho biên ?
 - Phải lưu toàn bộ tầng trước $\rightarrow O(b^s)$
- Đầy đủ ?
 - Nếu có giải pháp $\rightarrow s$ hữu hạn $\rightarrow$ Đầy đủ !
- Tối ưu ?
 - Nếu chi phí hành động = 1



So sánh: DFS và BFS



- Khi nào BFS tốt hơn DFS?
- Khi nào DFS tốt hơn BFS?

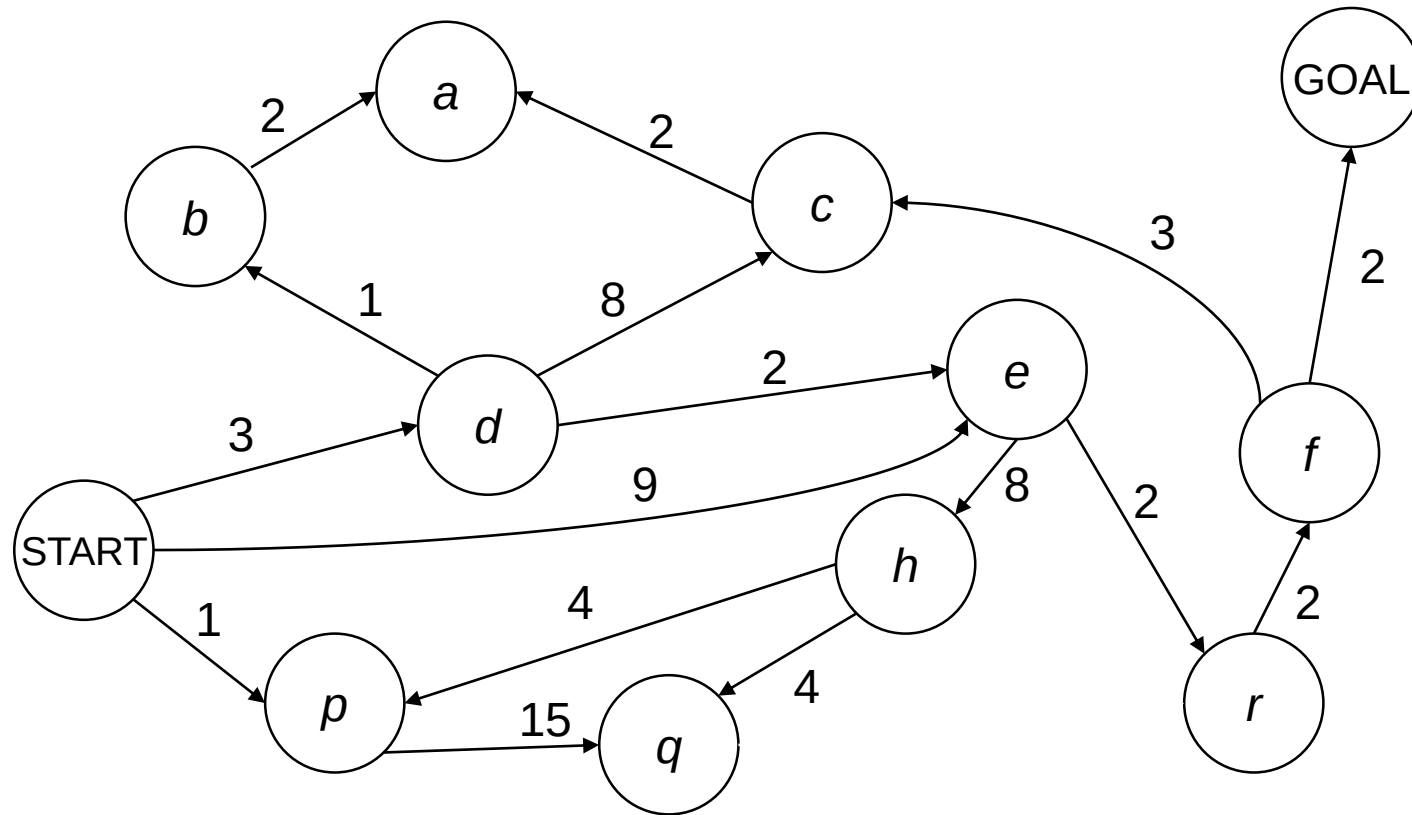
Ví dụ: Maze Water DFS/BFS (part 1)



Ví dụ: Maze Water DFS/BFS (part 2)



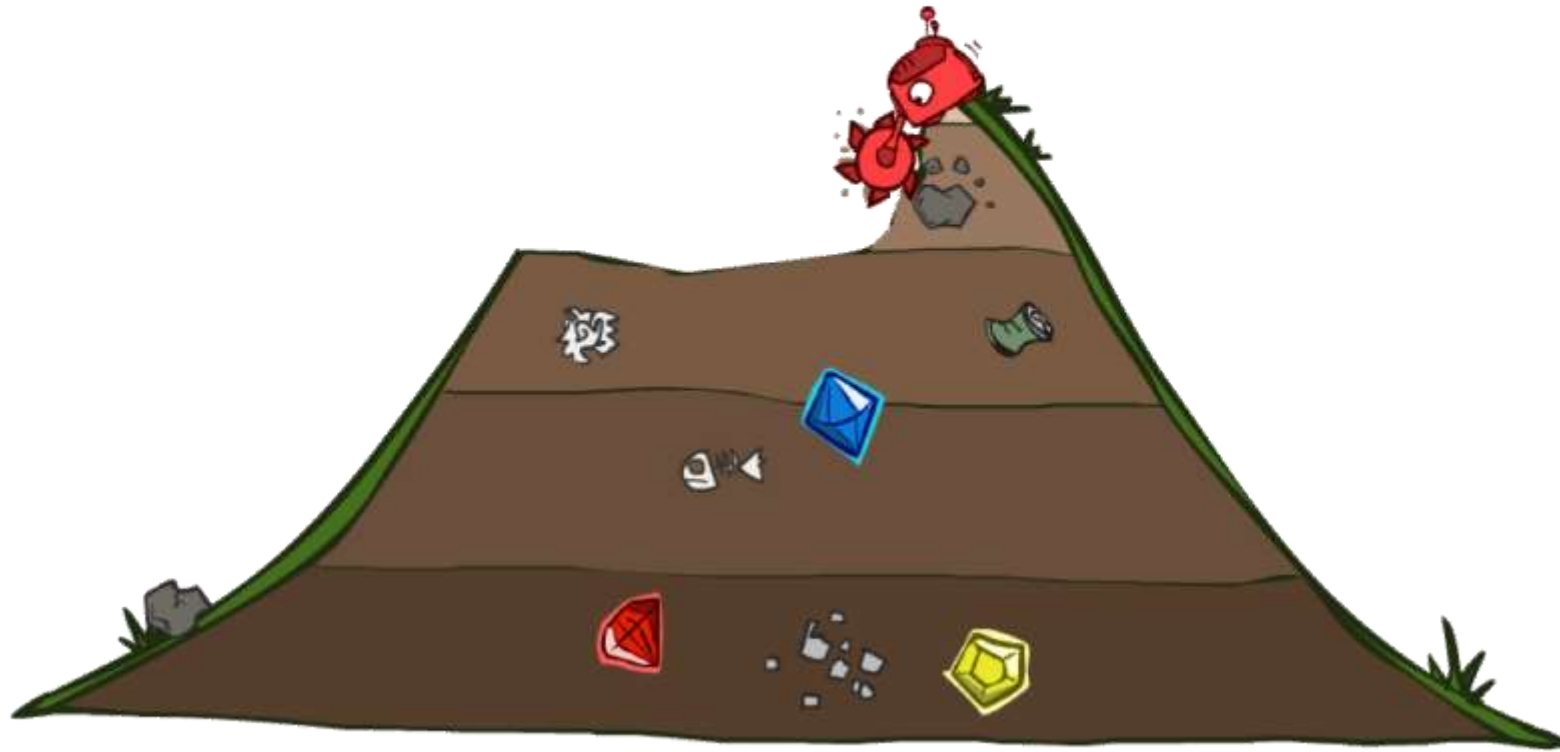
Tìm kiếm dựa trên chi phí



BFS tìm được kế hoạch ngắn nhất (theo số lượng hành động).
BFS không tìm kế hoạch có chi phí nhỏ nhất.

Tìm kiếm Chi phí đều

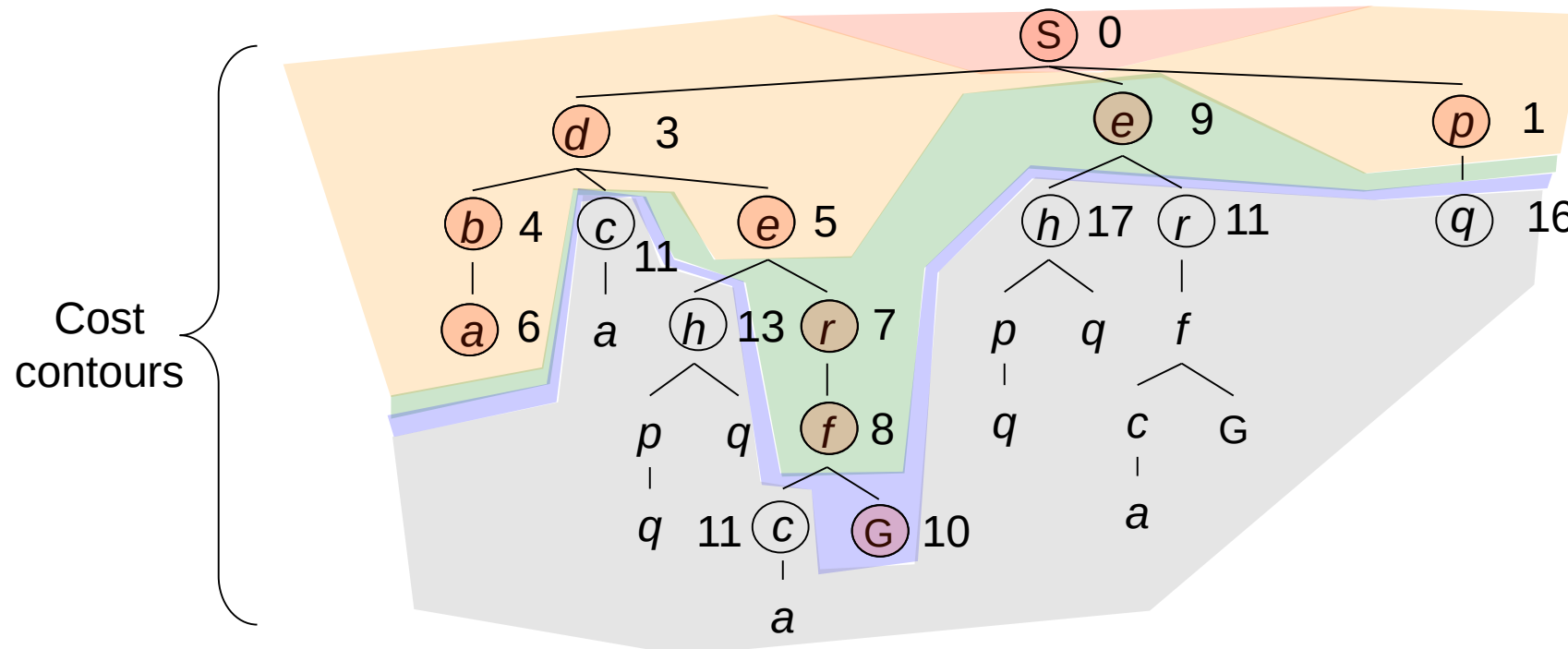
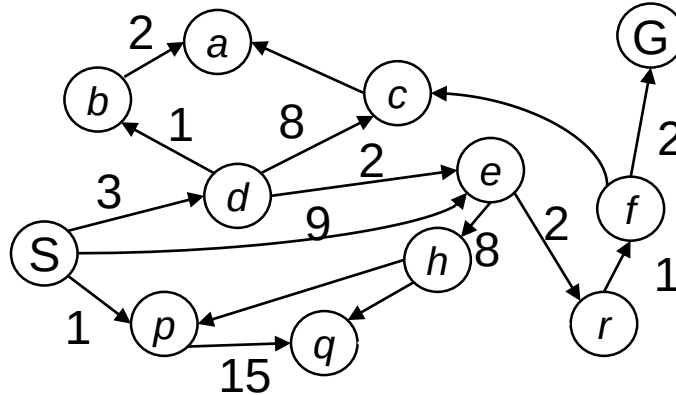
Uniform cost search - UCS



Tìm kiếm Chi phí đều (UCS)

Chiến lược: Mở rộng nút có chi phí nhỏ nhất

Cài đặt: Biên = hàng đợi ưu tiên (chi phí nhỏ nhất)



Tính chất của UCS

■ UCS mở rộng các nút nào ?

- Các nút có chi phí thấp hơn giải pháp !
- Nếu giải pháp có chi phí C^* và hành động có chi phí thấp nhất là ε , thì độ sâu hiệu quả là C^*/ε
- Độ phức tạp thời gian $O(b^{C^*/\varepsilon})$

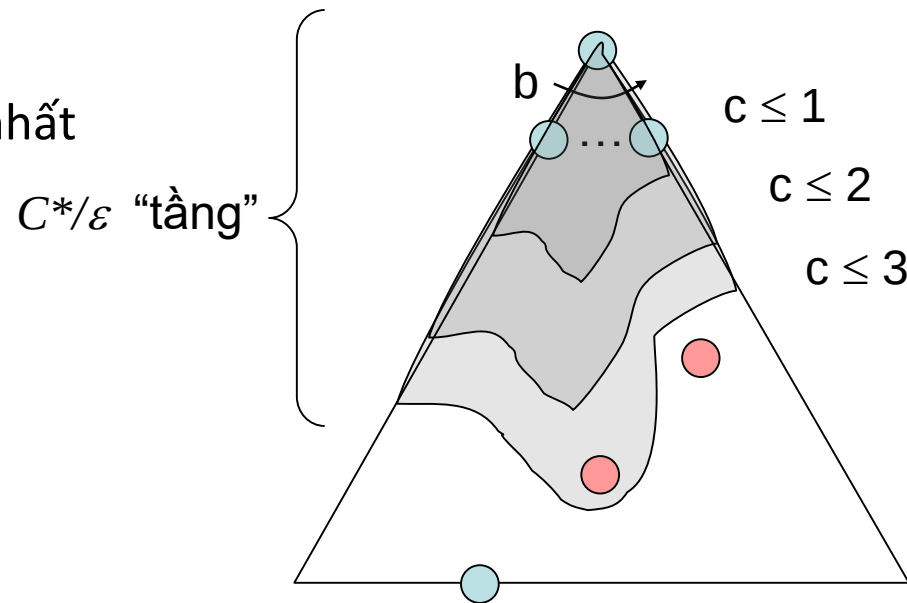
■ Bộ nhớ cho biên ?

- Phải lưu toàn bộ tầng trước $\rightarrow O(b^{C^*/\varepsilon})$

■ Đầy đủ ?

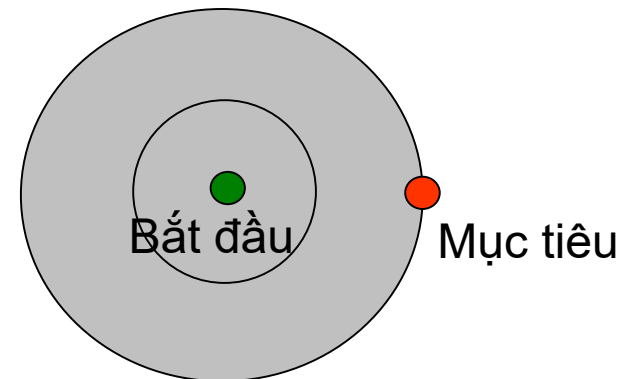
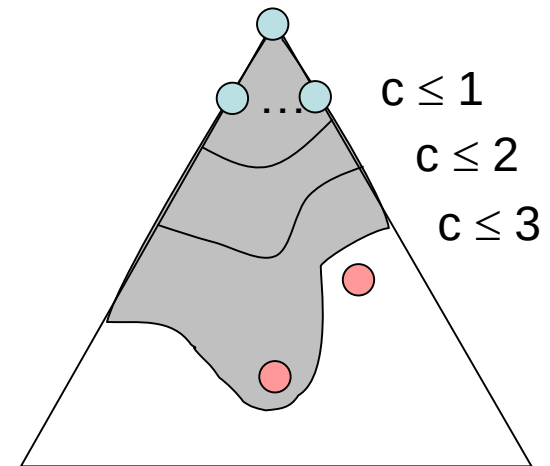
- Nếu giải pháp tối ưu có chi phí hữu hạn và hành động có chi phí dương $\rightarrow$ Đầy đủ !

■ Tối ưu ?



Các vấn đề của UCS

- UCS khám phá cây tìm kiếm theo đường đồng mức của chi phí
- Điểm tốt: UCS đầy đủ và tối ưu !
- Điểm xấu:
 - Khám phá các kế hoạch theo mọi hướng có thể
 - Không sử dụng thông tin về mục tiêu
- Chúng ta sẽ khắc phục điểm này !



Ví dụ: UCS môi trường rỗng



Ví dụ: Deep/Shallow Water --- DFS, BFS, hay UCS? (3)



Ví dụ: Deep/Shallow Water --- DFS, BFS, hay UCS? (1)



Ví dụ: Deep/Shallow Water --- DFS, BFS, hay UCS? (2)



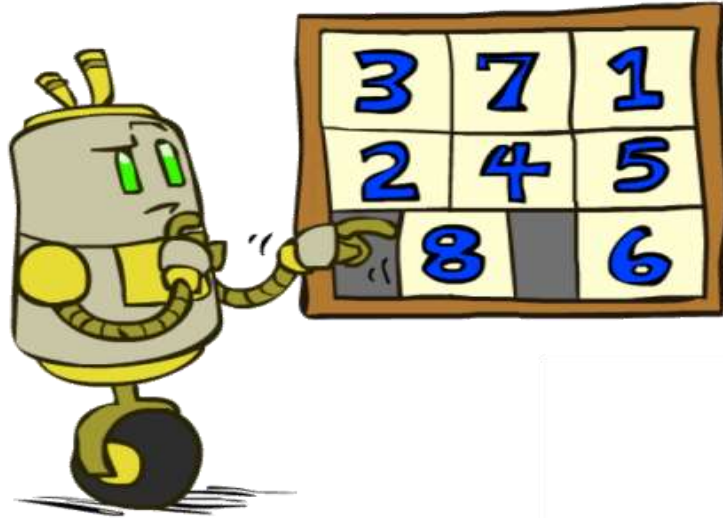
Tổng kết

- Bài toán tìm kiếm, không gian trạng thái, cây tìm kiếm
- Các thuật toán DFS, BFS, UCS
 - Có thể thấy các thuật toán khác nhau ở xác định “biên tìm kiếm”
 - Có thể cài đặt một cách tổng quát với biên là một hàng đợi ưu tiên
- Các thuật toán này tìm kiếm theo mọi hướng, không có hoặc không sử dụng thông tin về mục tiêu nên còn gọi là các thuật toán “tìm kiếm mù” (uninformed search)

Bài tập

- Cài đặt các thuật toán DFS, BFS, UCS
- Liên hệ giữa thuật toán UCS và thuật toán Dijkstra
- Viết chương trình giải trò chơi đẩy ô số

7	2	4
5		6
8	3	1



	1	2
3	4	5
6	7	8